卒　業　論　文

卒論タイトルなんたらかんたら

— 副題ああだこうだ —

20yyMTxxx　南山 太郎
20yyMTxxx　南山 花子

指導教員　志度 巨樹

20yy 年 mm 月

南山大学 理工学部 ソフトウェア工学科

Title Foo and Bar

— Subtitle Bar or Foo —

20yyMTxxx　NANZAN Taro
20yyMTxxx　NANZAN Hanako

Supervisor　SIDO Kyoju

Month 20yy

Department of Software Engineering
Faculty of Science and Engineering
Nanzan University

要約

ここに本論文の要約を書く。要約は論文のエッセンスを抜き出したものであるので、ここで取り扱う問題、その問題を解決するための手法、および主な成果が書かれていなければならない。要約を読むだけで、論文の概要が分かり、読者にとって興味を抱く内容か否かが分かるようになっている必要がある。

日本語と英語の両言語で要約を書くが、必ずしも 1 対 1 に対応する文章になっている必要はない。それぞれにふさわしい表現があるからである。

Abstract

In this part, the abstract of this paper is described. Since 'abstract' means the essence or summary of the paper, the abstract should include the description on the problems treated in the paper, the author's approach for solving those problems, and the main results. Readers will understand the outline of the paper, without reading other parts, and be able to decide whether they will have interests in the paper or not.

目次

第 1 章　はじめに 1
- 1.1 概要 1
- 1.2 目的 1
- 1.3 貢献 1
- 1.4 構成 1

第 2 章　背景 2
- 2.1 エッジデバイスの普及と現状 2
- 2.2 エッジデバイスの発熱 2
- 2.3 発熱による不都合 2
- 2.4 エッジデバイスにおける冷却機構 2
- 2.5 現在 2
- 2.6 課題点 3
- 2.7 アプローチ 3

第 3 章　設計 4
- 3.1 システム設計 4
- 3.2 システムの概要 5
- 3.3 動作原理 5
- 3.4 評価指標 5

第 4 章　実装 7
- 4.1 実験環境 7
- 4.2 実験 7

第 5 章　実験結果 8
- 5.1 スループット 8
- 5.2 温度変化 8
- 5.3 優先実行タスクの割合とスループットの関係 9

第 6 章　おわりに 10

参考文献 11

付録 A　プログラムリスト 12

付録 B　実行例 12

第 1 章

はじめに

1.1 概要

近年、IoT の普及に伴いエッジコンピューティングの重要性が急速に高まっている。Raspberry Pi 5 などのエッジ端末は監視カメラの画像処理やセンサーデータのリアルタイム解析など、高度な演算処理を実行するために広く利用されている。しかし、それらの機器は冷却機能が簡易的であることが多く、高負荷時には CPU 温度が上昇することによるサーマルスロットリングが発生し、システム全体の処理能力が著しく低下するという課題がある。本研究では，Linux カーネルの拡張機能である sched_ext を用いてタスクの重要度と CPU 温度に基づいた動的なスケジューリング手法を実装し, 体感を損なわない応答時間で運用可能であることを示す.

1.2 目的

本研究の目的はタスクの実行頻度を重要度に基づいて変更することで、温度と最低限のスループットの低下の両立を示すものである。

評価においては作成したプログラム実行時と何も実行していないデフォルトの状態でそれぞれ同じ負荷をかけ、スループット、温度の推移を比較する。

1.3 貢献

本研究の貢献は以下の点である.

- カーネル内で完結する高温防止制御を提示し, 試作を行う
- 試作したシステムを用いて温度, 消費電力, 応答時間とスループットを評価する

1.4 構成

本論文の構成は以下の通りである。

第 2 章では

第 3 章では

第 4 章では

第 5 章では

第 6 章では

第 2 章

背景

2.1　エッジデバイスの普及と現状

近年、データ処理を中央のクラウドではなく、データの発生源近くで行うエッジコンピューティングが注目を集めている。従来のクラウド中心のシステムでは高性能なサーバで処理できる代わりに膨大な量のデータを遠隔地に送信する必要があったり通信におけるセキュリティが問題視されていた。しかし、エッジデバイスの処理能力の向上に伴い、従来はクラウド側でしてきた処理をエッジが担う機会が増えてきた。これにより、リアルタイム性、プライバシー保護の両方を担保することが可能になった。

2.2　エッジデバイスの発熱

エッジデバイスの処理能力の向上に伴い、発熱量も増加している。大規模な演算を実行する過程で大量の電力を消費し、電力による熱生成が発生するためである。集積度が高いデバイスや小型なデバイスほど単位面積あたりの熱発生量が増大し、発熱が顕著になる。

2.3　発熱による不都合

エッジデバイスの発熱は単なる温度上昇にとどまらず、システムの性能やハードウェアの劣化を引き起こす。具体的には

1. 強制的に CPU 周波数の上限を落とすことで消費電力そのものを抑える
2. ハードウェアの劣化温度が 10 ℃上昇すると故障率が倍になるアレニウスの法則

高い周波数を維持することはそれだけ早くタスクを完了させられることでもあるが、同時にエッジデバイスの寿命を削る行為にもなり得る。この様に温度上昇は信頼性や安全性の面で無視できないリスクを伴う。

2.4　エッジデバイスにおける冷却機構

エッジデバイスで主流な冷却方法はハードウェア、ソフトウェアの両方で行うことができる。
ハードウェア的手法ではヒートシンクなどの高温側から低温側に熱が直接伝得る方式とファンなどの空気が動くことで熱を外部へと運ぶ方式の 2 種類がある。
ソフトウェア的手法だと温度抑制の手法として DVFS(Dynamic Voltage and Frequency Scaling) を用いたものが標準的である。これは CPU 温度が一定の値を超えると CPU 周波数を強制的に低下させることで発熱を抑えるというものである。

2.5　冷却機構の課題点

様々な冷却機構が導入されているが、技術的な限界や使用環境によって導入できない方法なども存在する。例えば、静音性や塵、埃などの観点からファンを持たない機器が求められたり、ヒートシンクなどもサイズやコスト、高出力熱源に対応しきれないなどの問題がある。また、DVFS はシステム全体の性能を一律で抑制することになるため、ユーザーにとって大切なタスクの処理も遅延してしまうことになる。エッジデバイスに求められるリアルタイム性が損なわれたり、そもそもこういったスロットリングはかなりの高温時における緊急的な意味合いが強いので、先述の信頼性や安全性を損なってしまう。

2.6　アプローチ

したがって、エッジデバイスの長期信頼性を確保するためには平均温度を低く保つようなスケジューリング制御が必須となる。
以上の課題に対し、本研究では温度を下げるために全タスクを一律に遅くするのではなく、タスクの重要度に応じて実行頻度を調整することで重要なタスクの応答を守りながら温度上昇を抑えることを試みる。

第 3 章

設計

3.1 システム設計

本章では先程の課題の解決案を述べる。

3.1.1 使用技術

以下に本研究で使用する技術について名称とその特徴を記載する。

eBPF(Extended Berkeley Packet Filter)
: カーネル空間にプログラムを安全に差し込むことで機能を拡張できる基盤
: オーバーヘッドが極小

SCX(sched_ext)
: Linux カーネル標準のスケジューラ機能を拡張するフレームワーク
: eBPF で定義可能
: 一定時間実行されないタスクがあると自動でスケジューラを落としてデフォルト状態に切り替える

本研究のアプローチの手段として SCX を選定した。
理由としては容易にスケジューラの差し替えが可能な点、eBPF を基にしているので安全性が担保されている点、オーバーヘッドを抑えられる点である。
また、以下に本研究で登場する eBPF や SCX の関数や概念について記す。

フックポイント
: カーネル関数の入口、出口、観測点を監視可能

DSQ(Dispatch Queue)
: タスクの待機場所
: FIFO か優先度キューとして扱う

ローカル DSQ
: 各 CPU 専用の待ち行列

グローバル DSQ
: どのローカル DSQ からもアクセス可能な DSQ
: システム全体で 1 本

カスタム DSQ
: ユーザーが任意本数定義可能なローカル DSQ に入る前のタスクの待機場所
: FIFO、優先度キューのどちらかで定義可能

select_cpu **コールバック**
: タスクが実行可能状態になると呼ばれる

対象のタスクをどのコアで処理するかの選定を担う

enqueue コールバック

select_cpu の後に呼ばれる

対象のタスクをグローバル DSQ かカスタム DSQ のどちらかに入れる

dispatch コールバック

CPU がタスクを欲した時に呼ばれる

enqueue コールバックで挿入したタスクをローカル DSQ に移す

なお、本研究ではカスタム DSQ はカーネル専用、重要タスク用、一般タスク用の 3 本使用する。名前はそれぞれ KERNEL、FAST、NORMAL とする。

3.2 システムの概要

温度監視は ebpf のフックポイントである tracepoint/thermal/thremal_temperature を常時参照する。
温度は 3 つの段階で制御する。状態遷移にはヒステリシスを設け、頻繁な状態の変動を防止している。各カスタム DSQ から取り出す量は温度の段階によって変化する。ただし、カーネルタスクは温度に左右せずにローカル DSQ へ移動させる。これはカーネルタスクが詰まることで必要な処理が遅れてしまったり、それによってスケジューラが落ちてしまうことの対策である。

3.3 動作原理

以下にタスクがどのような流れでカスタム DSQ に挿入され、どのような流れでローカル DSQ に挿入されるのかのフローを示す。

タスクがカスタム DSQ に入るフロー

1. タスクが実行可能状態になる
2. select_cpu コールバックが呼ばれる
   (a) どのコアで処理するかの決定
3. enqueue コールバックが呼ばれる
   (a) カーネル用のタスクであれば KERNEL へ移動
   (b) それ以外で指定した要件を満たす場合もしくは宣言されている場合は FAST へ移動
   (c) それ以外は全て NORMAL へ移動

タスクが処理されるフロー

1. CPU がタスクを欲する
2. dispatch コールバックが呼ばれる
   (a) 現在の温度ステージに基づき処理させるタスク量を決定
   (b) カーネルタスクは無条件でローカル DSQ へ移動
   (c) その他のタスクは各群から決められた分だけ select_cpu で決められたコアのローカル DSQ へ移動

なお、FAST に挿入されるタスクは締切が近いタスクと指定した PID のタスクの 2 種類がある。これは本研究ではどの程度の割合が優先的に処理されるのが最も効率が良いかを測る意図がある。

3.4 評価指標

本研究では作成したプログラム実行時とデフォルトの状態でそれぞれ同じ負荷をかけ、そのスループットと温度推移を評価する。

3.4.1 スループット測定

負荷をかけるにあたって、効率よく負荷をかけられる stress-ng コマンドを採用した。100 個の負荷を子プロセスで展開して、全て実行可能になったことを確認してから一斉に実行する。実行してから全ての子プロセスが終了するまでの時間を計測する。
負荷軽減プログラム実行時は実行前に指定した割合だけ FAST へ登録する。

3.4.2 温度推移測定

1 つの負荷を約 15 分間に渡ってかけ続け、その間の温度を測る。

第 4 章

実装

本章ではシステムの実装時の環境や実験手順に浮いて述べる

4.1 実験環境

実験に使用したハードウェア及びソフトウェア環境を以下表に示す。

表 4.1 実験環境

使用機器	Raspberry Pi 5 Model(メモリ 8GB)
バージョン	Linux raspberrypi 6.12.34-v8-16k
eBPF バージョン	
SCX バージョン	

4.2 実験

第 5 章

実験結果

5.1　スループット

デフォルト時とプログラム実行時の各温度ステージのスループットは以下図のようになった。

5.2　温度変化

作成したプログラムを動かしている時とそうでないときで同じ負荷をかけ、温度変化を監視した。以下にその結果を示す。
また、以下表に平均値、中央値、最高値を示す。

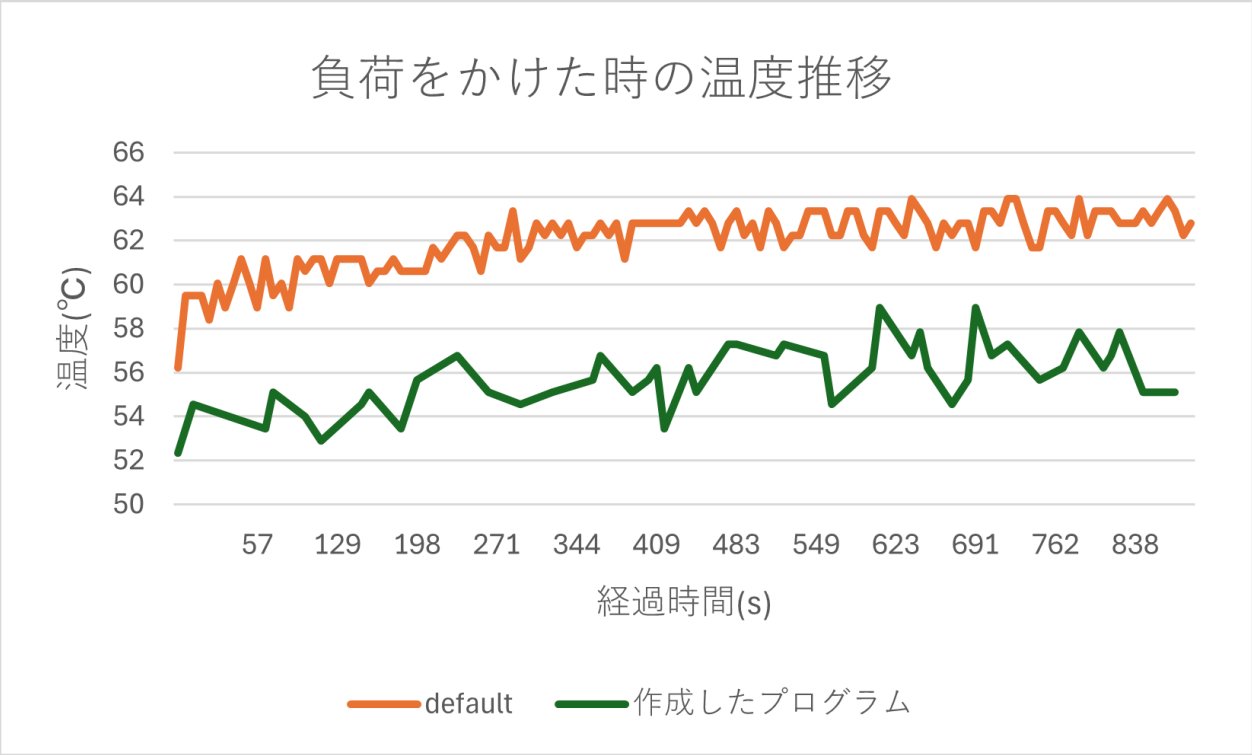

図 5.1　温度推移

表 5.1　実験結果

	デフォルト時	プログラム実行時
平均温度	62.02	55.78
中央値	62.25	55.65
最高温度	63.9	58.95

5.3　優先実行タスクの割合とスループットの関係

優先タスク群に登録される割合を 5% ずつ増やしていくと以下図のようになった。１０〜２０%あたりが最も効率が良くなるばらつきがあるのは実行時に負荷以外のタスクがどれだけ動いているかで変わってくるためである。

第 6 章

おわりに

本研究をまとめると、こういうことだったのだ。こうした結果になったのは、この点が良かったと思われる。しかし、あの点については、ああいうことも検討する必要があり、今後の課題として残っている。

謝辞

本研究を進めるにあたり、筆者は何もしてこず、ひとえに周囲の皆様の手とり足とりのご指導によるもので、関係諸氏に深く感謝します。

参考文献

[1] 愛 上男：“プログラミング論”, 情報処理論文誌, Vol.1 No.1, pp.1-10 (1991.1).
[2] Z.MrX：to Make Programming Much Easier, `http://www.abc.def.gh/ijkl.html`.
[3] 柿 久華子：“万能方法論”, 日本出版 1990.

付録 A

プログラムリスト

付録 B

実行例

実行例を示す。

```
Hello, world
```